First CUDA applications

Course: Massively Parallel Programming on GPUs

Teacher: Donato D'Ambrosio
Master Degree in Computer Science
Department of Mathematics and Computer Science
University of Calabria, Italy

October 16, 2026

1 Introduction

This first CUDA lab consists on:

- Learning some new aspects of the CUDA architecture by reading three short articles;
- Developing several implementations of the vector addition application;
- Developing several implementations of the matrix addition application;

2 Activity 1 - Learning

Read the following articles about:

- what is and how to use the CUDA Unified Memory;
- what is and how to use the Grid-Stride Loops kernel execution model;
- what is and how to use error chcking in CUDA
- what is and how to use a CUDA profiler
- nvprof (compute capability $<$ 8.0 - e.g., GTX 980)[1]
- ncu (compute capability $>=$ 8.0 - e.g., RTX 3050)[2]

3 Activity 2 - Vector Addition

Let A and B be the vectors of double-precision floating-point numbers to be added, C the resulting vector, and n = 2^{24} be their sizes. Implement a serial version of the vector addition application in C (for comparison). Implement four versions of the vector addition application in CUDA using:

- Unified-Memory/Monolithic-Kernels;
- Unified-Memory/Grid-Stride-Loop-Kernels.
- Standard-Memory/Monolithic-Kernels;
- Standard-Memory/Grid-Stride-Loop-Kernels.

  In the CUDA implementations, the A, B and C buffers must be initialized GPU-side by the `vectorInit()` kernel. Three calls to vectorInit are needed.

[1] Parameters to be used in subsequent labs are reported in Table 1 here.
[2] Parameters to be used in subsequent labs are reported in Table III here.

All the elements of A must be set to 1, all the elements of B to 2, all the elements of C to 0. The calculation must be also done on the GPU by defining the `vectorAdd()` kernel within the application.

Assess the performance of the serial implementation by considering the elapsed times of the call to vectorInit and vectorAdd (computatioanl part only) on one core of the JPDM2 workstation.

Assess the CUDA implemetataions on one of the Nvidia GTX 980 GPU installed on the JPDM2 workstation using blockSize = 32, 64, 128, 256. 512 and 1024, by summing the elapsed times of the host-device memory copies, the three calls to matrixInit and the call to matrixAdd through nvprof.

Check for the correctness of the CUDA implementations. Correctness is fundamental!

4 Matrix Addition

Repeat the steps from Section 3 for the matrix vector problem, applying them to the matrix addition problem. For the dimensions of the matrices, consider (nrows, ncols) = (n/2, n), being n the number of vector elements in the vector addition problem.